From Data to Action: Designing Human-Centered, Impact-Driven Decision Systems

Assignment 2

Due Date	Submission
September 18, 2026 11:59PM	Gradescope

This assignment is worth *100 points*, accounting for **15%** of the total course grade. It consists of **3 problems** aligned with techniques/tools covered in **lectures in Weeks 3-4**. Each problem may have one or two components: **written** and **programming**. You are expected to first work through the written questions to build context and conceptual understanding, and then, if applicable, complete the programming part.

Please strictly adhere to the Academic Integrity policies stated in the course syllabus and DO NOT CHEAT. We will run plagiarism checks on Gradescope.

In general, the following is required:

- Follow Submission Instruction *(2 points)*
- Problem #1: Shortest Path via Backward Induction *(35 points)*
- Problem #2: Hospital Surge Staffing via Q-Learning *(35 points)*
- Problem #3: Ride-hailing Pricing via Decision-Dependent Predict-then-Optimize *(28 points)*

Submission Instruction

Gradescope will have **two submission entries**: one for the programming and the other for the written. Failing to follow the submission instructions will result in a 2-point deduction as well as a delay in the grading process.

Programming Submission

One programming submission attempt should include **all Jupyter Notebooks and, if required, CSV files**. Submission attempts are unlimited. The Autograder relies on file names to run and assign grades. Therefore, please do **NOT rename** the provided start notebooks. If the handout specifies a required name for an output file, you must **use that exact same name** so the Autograder can successfully access it.

Written Submission

The written submission should be **1 single PDF file** containing your answers to all written questions. It must strictly follow this handout template without altering its layout or page structure. In other words, your PDF submission should have the same number of pages and the same alignment as this provided PDF template. Misalignment will cause Gradescope to incorrectly detect answer positions, which you can notice right after submission; as a result, your submission may not be graded correctly by the AI-assisted Autograder.

To ensure page consistency and alignment, you may complete the written questions in one of the following ways:

- type in LaTeX using the provided `.tex` file (we recommend using Overleaf)
- write on an electronic handout directly
- print the handout, write by hand, and scan it at Letter size

Each written question is either a multiple-choice question (MCQ) or a short-answer question (SAQ). You may receive partial credit on "Select all that apply" MCQs. Specific answer instructions are as follows:

For "Select one" questions, please fill in the appropriate circle completely:

1. (1 pt) **Select one:** Who teaches this course?

   ● Peter Zhang

   ○ Michael Johnson

   ○ Jennifer Smith

- If you work in LaTeX, simply replace `\choice` with `\CorrectChoice` as your answers. Our pre-defined environment will render PDF correctly.
- If you write on an electronic copy, use **black** color to fill the circle. An overfilled circle is acceptable.
- If you write on a print copy and need to change your answer, you may cross out the previous answer and circle in the new answer:

  (1 pt) **Select one:** Who teaches this course?

  ● Peter Zhang

  ○ Michael Johnson

  ⊗ Jennifer Smith

For "Select all that apply" questions, please fill in all appropriate squares completely:

2. (2 pts) **Select all that apply:** Which ones are the correct course number?

   ■ 94-867

   ■ 19-867

   ■ 12-768

   □ 12-867

   □ None of the above

- If you work in LaTeX, please do **NOT modify** the following line, which ensures square option symbol:
  `\checkboxchar{$\Box$} \checkedchar{$\blacksquare$}`
- To change answers on a print copy, you may cross out the previous answer(s) and fill in the new answer(s), the same as "Select one".

For "Short answer" and "Show your plot" questions, please complete inside the corresponding answer boxes. Two sample answers are shown below: one is acceptable, while the other is NOT.

3. (5 pts) **Short answer:** Use the `lipsum` package to generate a paragraph of Lorem Ipsum text

> Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

↑ This answer is **acceptable**.

(5 pts) **Short answer:** Use the `lipsum` package to generate the first paragraph of Lorem Ipsum text.

> Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

↑ This answer is **NOT acceptable**.

For "Show your code" questions, the general requirements stay the same as "Short answer" and "Show your plot". When asked to show code for a function, your response must include the complete function definition with all lines of code (an example below for your reference). Note that pseudocode will NOT receive any credits.

4. (5 pts) **Show your code:** Complete the `show_all_entry()` function.

```
def show_all_entry(array):
    for r in range(array.shape[0]):
        for c in range(array.shape[1]):
            print(array[r, c])
```

You can use one of the following ways to show code:

- Type code in LaTeX. The given answer box is already pre-formatted for coding.
- Insert a screenshot of your code inside the answer box
- Write by hand with clear indents

1 Shortest Path via Backward Induction

The shortest path problem is a classic example in dynamic programming. It involves finding the most efficient route through a network where each step has an associated cost. Rather than solving the entire problem at once, dynamic programming breaks it into smaller stages, solving each subproblem and building up to the overall solution. This approach is especially useful when decisions at one stage affect future outcomes.

In this problem, you are asked to plan a trip from your home to one of the downtown parking lots. Nodes constitute the traffic network (Figure 1). The number in the **upper-left corner** of each node denotes its node ID and the number **inside the node** specifies the travel time required to pass through that node. **Your home is located at Node `A3`, while the parking lots are represented by any node in the rightmost column** (Nodes `F1-F6`). Each intermediate node corresponds to a neighborhood. The edges represent neighborhood boundaries, and crossing a boundary does not add any additional time.

The objective is to determine the **shortest path from your home to a parking lot** (any one of `F1-F6`). Although one could use forward calculation by enumerating all possible paths from the home node to a parking lot, it is computationally expensive. A more efficient method is to apply Backward Induction (BI) to the network.

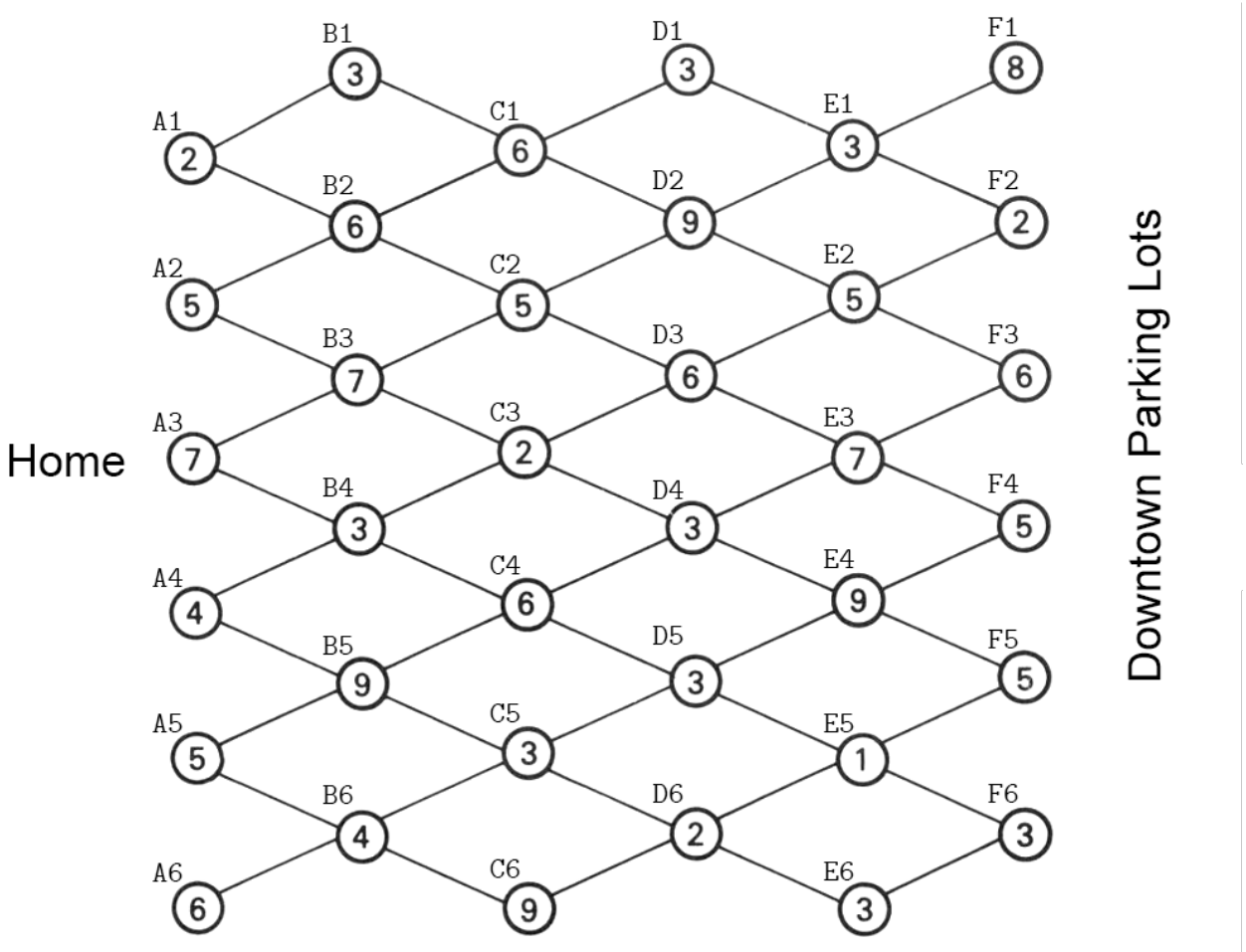


Figure 1: Path Network

The problem can be formulated as a dynamic decision process with multiple stages, where the six columns of neighborhoods represent the six stages, with the following notations:

- **Stage** $t \in \{1, 2, \ldots, 6\}$ represents the column in the network, where $T = 6$ is the terminal stage
- **State** s_t, with s_T as the terminal state and
- **Action** $a(s_t)$, e.g. $a(s_t = \texttt{A3}) = \texttt{B3}$
- **Transition Set** $N(s_t)$ is a set of all feasible next states that can be reached from the current state s_t
- **Cost Function** $g(s_t)$, e.g. $g(s_t = \texttt{B3}) = 7$
- **Cost-to-go Value** $J_t(s_t)$, with $J_T(s_T)$ for the terminal stage

1.1 Written

(a) *(5 points)* **Select all that apply:** Which of the following statements are correct?

- ☐ States refer to the specific node in the network at stages.
- ☐ Actions refer to the decision of which neighboring node to move to next.
- ☐ The number of states depends on the path chosen.
- ☐ The cost function measures the travel time.
- ☐ The cost-to-go value is the minimal travel time spent from home to the current state.

(b) *(5 points)* **Select all that apply:** Which of the following statements are correct?

- ☐ The first step of BI is to assign cost-to-go values to the home node.
- ☐ The first step of BI is to choose the parking lot with the lowest cost.
- ☐ BI computes the cost of all possible paths in this network.
- ☐ If two different paths have the same minimum total cost, BI considers either path as optimal.
- ☐ An optimal path must contain optimal sub-paths.

(c) *(5 points)* **Select all that apply:** Which of the following would affect the optimal path in this network?

- ☐ Close one of the parking lots
- ☐ Change travel time inside a single neighborhood
- ☐ Increase travel time in every node by the same constant
- ☐ Adding extra edges between neighborhoods
- ☐ Rename the node IDs

(d) *(5 points)* **Short answer:** Write the cost-to-go function *(Tip: formualte $J_t(s_t)$ and $J_T(s_T)$ separately).*

(e) *(10 points)* **Show your calculation:** Compute the cost-to-go value for node `D4`, $J_4(s_4 = \texttt{D4})$. Include the calculation steps from the terminal stage to stage 4. What is the action $a(s_4 = \texttt{D4})$?

(f) *(5 points)* **Short answer:** What is the optimal path of actions? Use the node ID.

2 Hospital Surge Staffing via Q-Learning

In this assignment, we will revisit the Hospital Surge Staffing problem discussed in the recitation.

As the following questions are based on the solution notebook, please study the notebook carefully before answering. You can find all `function()` and `class` mentioned in the questions from the notebook.

For your convenience, the solution notebook, named as `D2A_A2_HospitalStaffing.ipynb`, is included in the handout package. It is the recitation solution notebook posted on Canvas, with an added SARSA section at the end.

2.1 Written

(a) *(5 points)* **Select all that apply:** Which of the following statements about the Q-learning update rule (Equation 1) are correct?

$$Q_t(x_t, u_t) \leftarrow (1-\alpha)Q_t(x_t, u_t) + \alpha\Big(r_t + \gamma \min_{u_{t+1}} Q_{t+1}(x_{t+1}, u_{t+1})\Big) \tag{1}$$

- ☐ It blends old and new estimates using the learning rate α.
- ☐ It uses the immediate cost plus the discounted best estimate of future cost.
- ☐ It requires the full transition model of the environment to run.
- ☐ It is a sample-based approximation of the Bellman optimality equation.
- ☐ It ignores rewards or costs when updating.

(b) *(5 points)* **Select all that apply:** Which of the following statements correctly describe the role of ϵ-greedy exploration in Q-learning, i.e., the `choose_action()` function of `QLearningAgent`?

- ☐ It prevents the agent from prematurely converging to a suboptimal policy.
- ☐ It guarantees the agent always finds the globally optimal solution.
- ☐ It encourages trying actions with uncertain or unexplored values.
- ☐ It balances the trade-off between learning new information and exploiting known information.
- ☐ The agent will still explore sufficiently after removing ϵ-greedy, because the learning rate α provides randomness.

(c) *(5 points)* **Select all that apply:** Which of the following statements distinguish Backward Induction from Q-learning?

- ☐ Backward induction requires full knowledge of the transition probabilities and costs.
- ☐ Backward induction computes the exact cost-to-go function, while Q-learning estimates it iteratively.
- ☐ Backward induction always converges faster than Q-learning, regardless of problem size.
- ☐ Q-learning learns optimal policies through simulated or real experience.
- ☐ Q-learning cannot achieve the optimal policy even with infinite training.

(d) *(2 points)* **Short answer:** When calling the `step()` function of `HospitalEnvQL`, one of the results is variable of next state. What does this next state variable include? Explain in words what it/they means/mean in the context of hospital surge staffing.

Q-table is one of the keys in tabular Q-learning. In the notebook, `QLearningAgent` initializes the Q-table using the code below, which is a 3-dimensional table.

```
self.Q = np.zeros((T+2, X_MAX+1, U_MAX+1), dtype=float)
```

(e) *(2 points)* **Short answer:** What does each dimension represent in the context of hospital surge staffing?

(f) *(2 points)* **Short answer:** Why is it `X_MAX+1` instead of `X_MAX` directly?

Learning rates decay over the course of learning in the notebook implementation. Therefore, we should formally write the update rule as Equation 2 where a_t varies over time.

$$Q_t(x_t, u_t) \leftarrow (1 - \alpha_t) Q_t(x_t, u_t) + \alpha_t \Big(r_t + \gamma \min_{u_{t+1}} Q_{t+1}(x_{t+1}, u_{t+1}) \Big) \tag{2}$$

(g) *(2 points)* **Show your code:** Which line of the `learn()` function of `QLearningAgent` in the notebook controls the learning rate decaying?

(h) *(5 points)* **Select all that apply:** Which of the following statements are correct?

- ☐ A high learning rate α always guarantees faster and more stable convergence.
- ☐ Decaying the learning rate α over time helps stabilize learning once Q-values are close to convergence.
- ☐ The discount factor γ controls how much future costs are taken into account relative to immediate ones.
- ☐ If $\gamma = 1$, future costs are considered equally important as immediate costs.
- ☐ If both γ and α are set to zero, Q-learning still converges to the optimal policy.

(i) *(2 points)* **Short answer:** The current a_t follows a time-dependent linear decay until it reaches a minimum threshold. How would you modify this schedule to use a non-linear decay instead? **Express your answer as a formula of the form** $a_t = \ldots$. Your proposed schedule could be either time-dependent or time-independent.

SARSA (State-Action-Reward-State-Action) is a simpler reinforcement learning algorithm than Q-learning. The agent works similarly to Q-learning that takes actions, receives feedback, and continuously updates its behavior to maximize long-term rewards.

SARSA updates the Q-value using Equation 3.

$$Q_t(x_t, u_t) \leftarrow (1 - \alpha_t)Q_t(x_t, u_t) + \alpha_t\Big(r_t + \gamma Q_{t+1}(x_{t+1}, u_{t+1})\Big) \tag{3}$$

As you can see, the key difference is that SARSA does not include min operation. Therefore, SARSA is an **on-policy** algorithm learning the value of the policy it is actually following, while Q-learning is **off-policy** that learns the value of the optimal policy while possibly following a different exploratory policy when taking actions.

(j) *(3 points)* **Select all that apply:** Which of the algorithms eventually converge to the same Q-values, despite differences in the number of steps required for convergence?

- ☐ Q-learning with ϵ-greedy, where $\epsilon = 0.1$
- ☐ SARSA with the same ϵ-greedy, $\epsilon = 0.1$
- ☐ SARSA with dynamic ϵ-greedy, where ϵ is gradually reduced from 0.1 to 0
- ☐ All three algorithms above lead to different Q-values

(k) *(2 points)* **Short answer:** Reason your answer to the above question.

3 Ride-hailing Pricing via Decision-Dependent Predict-then-Optimize

3.1 Programming

Complete the provided start Jupyter notebook, `D2A_A2_SurgePricing.ipynb`.

Export the following spreadsheets of your model results as guided in the notebook.

(a) *(10 points)* `prediction_test.csv`

(b) *(15 points)* `optimization_decisions.csv`

3.2 Written

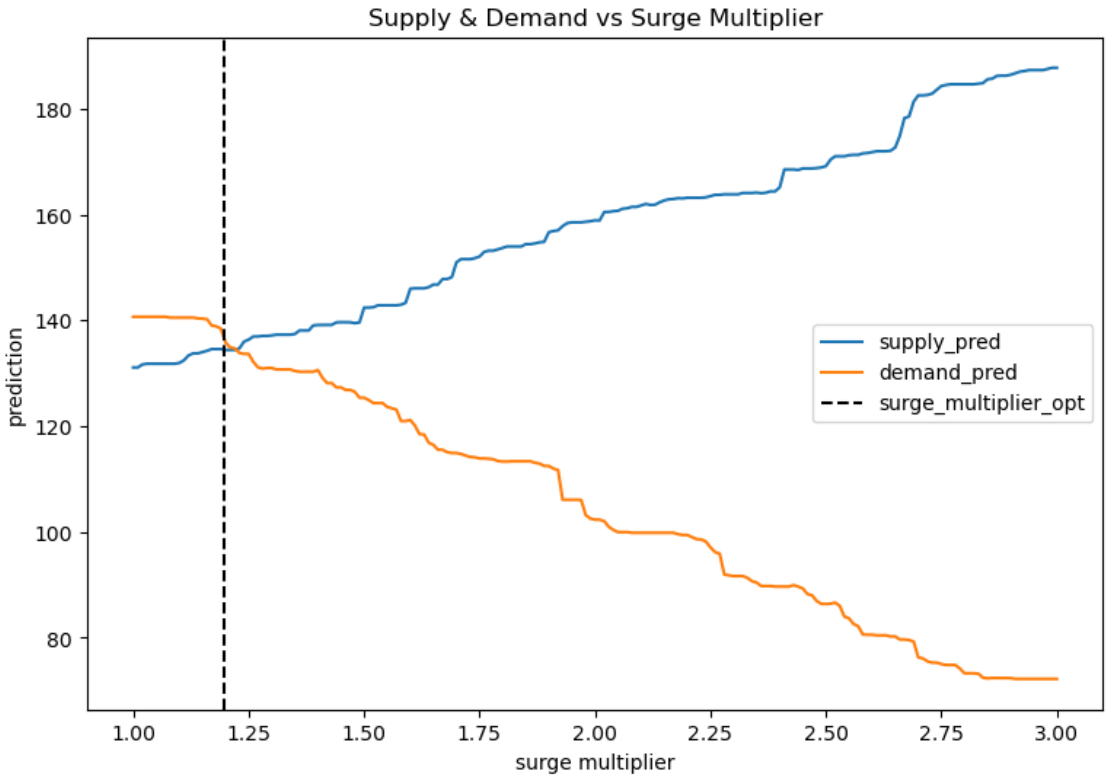


Figure 2: Sample Plot of the Last Cell of the Notebook

(a) *(3 points)* **Short answer:** If your notebook is implemented correctly, the last cell should output a figure similar to Figure 2. The figure includes the supply and demand curves. The black dashed vertical line indicates where the optimized surge multiplier is.
Carefully read the code and understand the plot. How do you interpret this plot?
(Answer box is on the next page)

Deliverables

Congratulations! All of the work is complete. The last step is to submit your deliverables to Gradescope:

- 1 single PDF file to "**Assignment 2 Written**"
- the follwoing files to "**Assignment 2 Programming**"
  - `D2A_A2_SurgePricing.ipynb`
  - `prediction_test.csv`
  - `optimization_decisions.csv`